QueueSmart – Assignment 4

Data Design & Database Implementation

Team: Agah Duzenli, Dang (wabisabiz), Skyler Tran, Derek Lihou · November 2026

1. GitHub Repository

Link: https://github.com/AgahDUZENLI/queuesmart (branch `main`)

For A4 we added a PostgreSQL database to QueueSmart. Everything the app uses is now saved in the database: accounts, profiles, places, services, queues, tickets and notifications. If the server restarts, nothing is lost.

The tables are defined in `backend/db/schema.sql`. The code that talks to the database is in `backend/src` (`db.js` and the `*Repo.js` files), and the tests are in `backend/tests`.

```
docker compose up -d     # starts the database, back end and front end
cd backend
npm install
npm test                 # runs the tests
npm run coverage         # runs the tests and shows coverage
```

Why PostgreSQL? Our data is connected: a place has services, each service has a queue, and each queue has tickets from users. A relational database fits this well. Foreign keys keep the links correct, and constraints stop bad data from being saved. It also fit our earlier work. In A3 all data went through one file, so we could replace it with database code without changing the API or the A2 screens.

2. Database Schema

We used every table from the assignment and added three small tables that the app needs.

Assignment table	Our table	Primary key	Foreign keys and notes
UserCredentials	`user_credentials`	`id`	Email is `UNIQUE`. The password is saved as a scrypt hash, never as plain text.
UserProfile	`user_profile`	`user_id`	`user_id` → user_credentials, `email` → user_credentials.email, `place_id` → place
Service	`service`	`id`	`place_id` → place
Queue	`queue`	`id`	`service_id` → service (one queue per service)
QueueEntry	`queue_entry`	`id`	`queue_id` → queue, `user_id` → user_credentials
Notification	`notification`	`id`	`user_id` → user_credentials
(extra)	`place`	`id`	The office or clinic an admin runs. Its 6-digit id is also the join code.
(extra)	`saved_place`	`user_id`, `place_id`	Each user's saved places. `user_id` → user_credentials, `place_id` → place
(extra)	`session`	`token`	Keeps users logged in. `user_id` → user_credentials

A few small differences from the assignment:

- A queue can also be `Paused`, not only Open or Closed, because our A2 admin screens already have a pause option.
- Instead of one "canceled" status, a ticket can be `left`, `removed` or `no-show`, so history shows why the ticket ended.
- When an admin deletes a service, we only mark it as deleted. This way users can still see their old tickets in history.

SQL to create the tables (backend/db/schema.sql)

```
-- QueueSmart database schema (PostgreSQL). Matches schema.dbml, plus the rules
-- DBML can't express: CHECK constraints and partial unique indexes.
-- Docker runs this file the first time the database volume is created.

CREATE TYPE user_role AS ENUM ('user', 'admin');
CREATE TYPE priority_level AS ENUM ('Low', 'Normal', 'High');
CREATE TYPE queue_status AS ENUM ('Open', 'Paused', 'Closed');
CREATE TYPE entry_status AS ENUM ('waiting', 'serving', 'served', 'left', 'removed', 'no-show');
CREATE TYPE notification_type AS ENUM ('Queue update', 'Status change');
CREATE TYPE notification_status AS ENUM ('sent', 'viewed');

CREATE TABLE place (
  id integer PRIMARY KEY, -- 6-digit id, also the join code
  name varchar(50) NOT NULL
);

CREATE TABLE user_credentials (
  id integer PRIMARY KEY, -- 6-digit id from the app
  email varchar(254) NOT NULL UNIQUE,
  password_hash varchar(255) NOT NULL, -- scrypt salt:hash, never plain text
  role user_role NOT NULL DEFAULT 'user',
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE user_profile (
  user_id integer PRIMARY KEY REFERENCES user_credentials (id),
  email varchar(254) NOT NULL UNIQUE REFERENCES user_credentials (email) ON UPDATE CASCADE,
  first_name varchar(50) NOT NULL,
  last_name varchar(50) NOT NULL,
  phone varchar(20),
  email_me boolean NOT NULL DEFAULT true,
  place_id integer REFERENCES place (id) -- admins only: the place they run
);

CREATE TABLE service (
  id integer PRIMARY KEY,
  place_id integer NOT NULL REFERENCES place (id),
  name varchar(100) NOT NULL,
  description varchar(500) NOT NULL,
  duration integer NOT NULL CHECK (duration BETWEEN 1 AND 480), -- minutes
  priority priority_level NOT NULL,
  max_in_line integer NOT NULL DEFAULT 25 CHECK (max_in_line BETWEEN 1 AND 500),
  opens time NOT NULL DEFAULT '09:00',
  closes time NOT NULL DEFAULT '16:30',
  appointments boolean NOT NULL DEFAULT false,
  deleted_at timestamptz, -- keeps ticket history after a delete
  CHECK (closes > opens)
);

CREATE INDEX service_place_id_idx ON service (place_id);
```

```
CREATE TABLE queue (
  id integer PRIMARY KEY,
  service_id integer NOT NULL UNIQUE REFERENCES service (id),
  status queue_status NOT NULL DEFAULT 'Closed',
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE queue_entry (
  id integer PRIMARY KEY,
  queue_id integer NOT NULL REFERENCES queue (id),
  user_id integer NOT NULL REFERENCES user_credentials (id),
  ticket varchar(12) NOT NULL, -- e.g. A-001
  service_name varchar(100) NOT NULL, -- name at time of joining
  priority priority_level NOT NULL DEFAULT 'Normal',
  position integer NOT NULL, -- queueOrder in the code
  status entry_status NOT NULL DEFAULT 'waiting',
  joined_at timestamptz NOT NULL DEFAULT now(),
  started_at timestamptz,
  completed_at timestamptz,
  close_notified boolean NOT NULL DEFAULT false
);

CREATE INDEX queue_entry_queue_status_idx ON queue_entry (queue_id, status);
CREATE INDEX queue_entry_user_status_idx ON queue_entry (user_id, status);

-- A user can only have one active ticket at a time
CREATE UNIQUE INDEX one_active_ticket_per_user ON queue_entry (user_id)
  WHERE status IN ('waiting', 'serving');

-- A queue can only serve one person at a time
CREATE UNIQUE INDEX one_serving_per_queue ON queue_entry (queue_id)
  WHERE status = 'serving';

CREATE TABLE notification (
  id integer PRIMARY KEY,
  user_id integer NOT NULL REFERENCES user_credentials (id),
  type notification_type NOT NULL,
  message varchar(255) NOT NULL,
  status notification_status NOT NULL DEFAULT 'sent',
  emailed boolean NOT NULL DEFAULT false,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX notification_user_id_idx ON notification (user_id);

CREATE TABLE saved_place (
  user_id integer REFERENCES user_credentials (id),
  place_id integer REFERENCES place (id),
  PRIMARY KEY (user_id, place_id)
);

CREATE TABLE session (
  token uuid PRIMARY KEY,
  user_id integer NOT NULL REFERENCES user_credentials (id),
  created_at timestamptz NOT NULL DEFAULT now()
);
```

3. Validation and Saving Data

We check the data in two places. First, the back end checks every request and sends back a clear error message. Then the database checks it again, so bad data can't be saved even if something gets past the back end.

Rule	Back end	Database
Required fields	Returns "… is required."	`NOT NULL`
Length limits	For example: names 50, email 254, description 500 characters	`varchar` with the same limits
Data types	Numbers, true/false, times and allowed values	Column types, enums and `CHECK` rules
Unique values	A duplicate email returns an error	`UNIQUE` email, and only one active ticket per user

All forms in the app send their data to the back end, which saves it in the database. The screens, like queue status, history, services, notifications and profile, load their data from the database. To test this, we restarted the server, and all the data was still there.

4. Code Coverage

We ran `npm run coverage` on November 6, 2026. All 95 tests pass, and coverage is 91.35%, which is above the 70–80% goal.

8 / 8	95 / 95	91.35%	81.26%	91.35%	94.23%
Test suites passed	Tests passed	Statements	Branches	Functions	Lines

`data.js` has low coverage because it is old code from A3 that the app no longer uses.

```
$ npm run coverage

------------------|---------|----------|---------|---------|-------------------------------------------------------------------
File              | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
------------------|---------|----------|---------|---------|-------------------------------------------------------------------
All files         |   91.35 |    81.26 |   91.35 |   94.23 |
 app.js           |   96.66 |       50 |     100 |     100 | 29-31
 auth.js          |    98.3 |    94.54 |     100 |   98.14 | 92
 data.js          |    37.5 |      100 |       0 |    37.5 | 13-21
 db.js            |    90.9 |       50 |     100 |     100 | 9
 demo.js          |     100 |      100 |     100 |     100 |
 history.js       |   90.38 |    76.31 |   84.21 |   90.69 | 47-50
 notifications.js |   91.11 |    84.21 |     100 |   93.02 | 83,106,139
 placeRepo.js     |   93.33 |    83.33 |     100 |     100 | 31
 places.js        |     100 |    92.85 |     100 |     100 | 32
 profile.js       |     100 |    91.66 |     100 |     100 | 21
 queueRepo.js     |   86.58 |    66.66 |   87.75 |   92.38 | 220,281,290-292,316,342-343,367,374,386-387,448-449,475-476
 queues.js        |      93 |    82.97 |     100 |   95.34 | 77,86,161,177
 serviceRepo.js   |   80.51 |     64.7 |   76.92 |   82.85 | 79-81,86,114-115,133-134,154-155,169-170
 services.js      |   94.02 |    76.47 |     100 |     100 | 57,77,92,96-107
 userRepo.js      |   98.46 |    91.66 |   93.33 |     100 | 82,146-147
 validation.js    |   98.41 |    98.55 |     100 |     100 | 42
------------------|---------|----------|---------|---------|-------------------------------------------------------------------

Test Suites: 8 passed, 8 total
Tests:       95 passed, 95 total
Snapshots:   0 total
Time:        4.156 s, estimated 5 s
Ran all test suites.
```

5. Team Contributions

Group member name	What is your contribution?	Discussion notes
Agah Duzenli (GitHub: AgahDUZENLI)	Designed the database tables (`schema.sql`) with their keys and rules, and added PostgreSQL to Docker so everything starts with one command. Tested the whole app with the database and wrote this report.	Nov 1: we picked PostgreSQL and split the work. I shared the schema and Docker setup. Nov 6: checked everything before submitting.
Skyler Tran (GitHub: skylertran19)	Moved user accounts and profiles to the database, and saved sessions too, so users stay logged in. Added a Profile page and set up the test database that all our tests use.	Nov 1: took accounts and profiles and finished them (PR #23).
Dang (GitHub: wabisabiz)	Moved services, queues, tickets, places and history to the database, and updated their tests.	Nov 3: finished services and queues (PR #24) and fixed a small queue bug.
Derek Lihou (GitHub: DerekLihou)	Moved notifications to the database. They are now saved and change to "viewed" when the user reads them. Added more notification tests.	Nov 5: finished notifications (PR #25).